

 RETRIEVAL SYSTEMS

# RAG Is No Longer the Only Pattern

5 retrieval systems — each solves  
a problem RAG can't

RAG

CAG

GraphRAG

Vectorless

Hybrid



AI Coach John

FOLLOW



 Repost

# Why RAG Alone Is No Longer Enough

## ! RAG'S HARD LIMITS (DISCOVERED IN PRODUCTION)

- > Fails when answer requires **connecting facts across many docs**
- > Fails when context window is large enough to **hold everything**
- > Fails when **structure matters** (entity relationships)
- > Adds unnecessary latency when **answer is already known**

## 📅 EVOLUTION TIMELINE

- 2020 Pure LLM — no retrieval
- 2022 RAG becomes the standard
- 2023 GraphRAG fills relationship gap
- 2023 CAG fills "just cache it" gap
- 2024 Vectorless RAG for structured data
- 2025 Adaptive routing across all patterns



AI Coach John

FOLLOW



🔄 Repost

# 5 Retrieval Patterns You Need to Know

| PATTERN           | HOW IT RETRIEVES              | WHEN IT WINS              |
|-------------------|-------------------------------|---------------------------|
| <b>RAG</b>        | Vector similarity search      | <b>General Q&amp;A</b>    |
| <b>CAG</b>        | Caches full context in window | <b>Stable, known docs</b> |
| <b>GraphRAG</b>   | Knowledge graph traversal     | <b>Relationships</b>      |
| <b>Vectorless</b> | LLM reasoning over SQL/DB     | <b>Structured data</b>    |
| <b>Hybrid</b>     | Routes across all patterns    | <b>Mixed workloads</b>    |



Each pattern solves a specific retrieval failure mode. The right choice depends on your data type and query patterns.



AI Coach John

FOLLOW



 Repost

# RAG — Retrieval Augmented Generation

## ⚙️ HOW IT WORKS

- > Chunk documents → embed → store in **vector DB**
- > At query time: embed query → cosine similarity → top-K chunks → LLM

```
// Ingestion
```

```
Docs → Chunk → Embed → Vector DB
```

```
// Query time
```

```
Query → Embed → Similarity Search → Top-K → LLM → Answer
```

## ✅ WHEN IT WINS

- > Large, dynamic knowledge bases
- > General Q&A with matching vocabulary
- > Unpredictable document needs

## ❌ LIMITATIONS

- > Precision limited by embedding quality
- > Fails on multi-hop queries
- > +100–400ms latency per query



# CAG — Cache-Augmented Generation

## ⚙️ HOW IT WORKS

Load **ALL relevant documents** into the LLM's context window — no retrieval step. Use KV cache to keep context warm across queries. LLM reasons over everything at once.

## 🕒 LATENCY COMPARISON



↓ 40% faster on warm cache

## ✅ WHEN IT WINS

- KB < 50 docs, rarely updated
- All queries need same context
- Latency-critical systems
- Highly interrelated documents

## ⚠️ LIMITATIONS

- Context window hard limit (~200K tokens)
- KV cache is expensive at scale
- Cache invalidation costly on updates



**Scenario:** Legal team with 30 contract templates. Every question uses the same 30 docs. CAG caches once, answers 1,000 questions with zero retrieval overhead.



AI Coach John

FOLLOW



↻ Repost

# GraphRAG — Knowledge Graph Retrieval

## 🔧 HOW IT WORKS

- Extract **entities & relationships** from docs during ingestion
- Build knowledge graph (nodes + edges)
- Traverse graph + vector search at query time
- LLM reasons over structured facts + chunks

## 🏗️ GRAPH STRUCTURE

```
[Company A]
  —invested_in→
    [Startup B]
      —acquired_by→
        [Company C]
[CEO John]
  —founded→ [Company A]
  —worked_at→ [Company C]
```

## 🏆 WHEN IT WINS

- Multi-document relationship queries
- Medical, legal, financial domains
- Global summaries across many docs

## ⚠️ LIMITATIONS

Expensive indexing (5–10× longer).  
Requires NLP entity extraction pipeline.  
Not for rapidly-changing KBs.

```
from graphrag import GraphRAGClient
client = GraphRAGClient(index_dir="./graphrag_index")
result = client.global_search("Main themes?")
result = client.local_search("Who is connected to Project Apollo?")
```



AI Coach John

FOLLOW



 Repost

# Vectorless RAG — LLM Over Structured Data

## 📌 WHAT IT IS

**No embeddings. No vector DB.** LLM generates SQL to query structured databases directly. Used when your data lives in tables, not documents.

```
from langchain.chains import create_sql_query_chain

chain = create_sql_query_chain(llm, db)

# NL → SQL
sql = chain.invoke({
    "question": "Avg order value for APAC enterprise clients Q3 2024?"
})

# → SELECT AVG(order_value) FROM orders
# WHERE region='APAC' AND tier='enterprise'
# AND quarter='Q3-2024'
result = db.run(sql)
```

## ✅ WHEN IT WINS

- > Data is structured (SQL, CSV, sheets)
- > Exact numbers matter (finance, metrics)
- > Queries need aggregation/filtering
- > Data updates too fast to embed

## ⚠️ LIMITATIONS

- > Only works on structured data
- > SQL hallucination risk
- > Requires schema understanding
- > No qualitative reasoning



# Hybrid Systems — Route to the Right Pattern

## HOW IT WORKS

A **query classifier** analyzes the query and routes it to the optimal retrieval pattern. Results from multiple patterns can be merged.

Query → Classifier

RAG

GraphRAG

SQL

CAG

↓ Result Merger ↓ LLM ↓ Answer

## EXAMPLE ROUTING RULES

→ **RAG** "What does our policy say about X?"

→ **GraphRAG** "How are X and Y connected?"

→ **SQL** "What was last month's revenue by region?"

→ **CAG** "Summarize all quarterly reports"



AI Coach John

FOLLOW



 Repost

# Head-to-Head: All 5 Patterns

| Metric           | RAG    | CAG       | GraphRAG  | Vectorless | Hybrid    |
|------------------|--------|-----------|-----------|------------|-----------|
| Latency          | Medium | Low       | Medium    | Low        | Varies    |
| Cost             | Low    | Medium    | High      | Low        | Medium    |
| General Q&A      | High   | Very High | High      | N/A        | Very High |
| Relationships    | Low    | Medium    | Very High | N/A        | Very High |
| Structured Data  | Low    | Low       | Low       | Very High  | Very High |
| Setup Complexity | Low    | Low       | Very High | Medium     | Very High |
| Scales with Data | Yes    | No        | Partially | Yes        | Yes       |
| Dynamic Data     | Yes    | No        | Partially | Yes        | Yes       |



# Real Latency Breakdown Per Pattern

**RAG** 1,250ms total



**CAG** 800ms total ✓ fastest



**GraphRAG** 1,400ms total



**Vectorless** 1,150ms total



**i** All timings exclude cold start. CAG wins only on warm cache. GraphRAG adds hours of indexing upfront.



**AI Coach John**

**FOLLOW**



**↻ Repost**

## \$ COST

# Cost Per 1,000 Queries (Approximate)



### 💡 KEY INSIGHT

CAG is more expensive per-query but **eliminates retrieval infrastructure**. For small, stable KBs it's often cheaper total cost of ownership. GraphRAG's per-query cost excludes the one-time indexing cost which can be substantial.



AI Coach John

FOLLOW



↻ Repost

# Pick Your **Retrieval Pattern**

**IS YOUR KB < 200K TOKENS AND MOSTLY STATIC?**

→ **YES:** Use **CAG** — simplest, lowest latency

↓ NO

**IS YOUR DATA STRUCTURED (TABLES, DATABASES)?**

→ **YES:** Use **Vectorless RAG** (Text-to-SQL)

↓ NO

**DO QUERIES NEED RELATIONSHIP TRAVERSAL?**

→ **YES:** Use **GraphRAG**

↓ NO

**MIXED QUERY TYPES?**

YES → **Hybrid**

**OTHERWISE**

→ **Standard RAG**



**Rule:** Start with the simplest pattern that can work. Complexity compounds with every component you add.



**AI Coach John**

**FOLLOW**



 **Repost**

# Retrieval Patterns Cheatsheet

| PATTERN           | USE WHEN                                  | AVOID WHEN                            |
|-------------------|-------------------------------------------|---------------------------------------|
| <b>RAG</b>        | Large dynamic KB, general Q&A             | Relationship queries, structured data |
| <b>CAG</b>        | Small stable KB, all-docs-always-needed   | Large KB, frequent updates            |
| <b>GraphRAG</b>   | Relationship queries, multi-doc reasoning | Simple Q&A, budget-constrained        |
| <b>Vectorless</b> | Structured data, exact numbers needed     | Unstructured text, qualitative Qs     |
| <b>Hybrid</b>     | Mixed query types, production scale       | MVP/prototype, limited eng capacity   |

💡 **RAG is the default — but not always right.**  
Know which pattern fits your query type, then build.



AI Coach John

FOLLOW



↻ Repost